SPURVANCE LABS

Data Integrity & Cryptography Basics

Individual Assignment Report

Project: SentinelHash — File Integrity Monitor (Kali Linux)

Prepared by	Muhammad Saad Shakeel
Student ID	SPL-2026-0433
Mentor	Samiullah
Date	27/10/2026
Environment	Kali Linux terminal

Table of Contents

1. Introduction 3
1.1 Report structure 4
1.2 Scope and authorization 4
2. Part A: Core Concepts 5
2.1 Data integrity 5
2.2 Hashing 5
2.3 Encryption 5
2.4 Digital signatures 5
2.5 Summary comparison 5
3. Part B: SHA-256 vs SHA-1 vs MD5 6
3.1 Why the older algorithms fail 6
3.2 Recommendation 6
4. Part C: SentinelHash Mini-Project 7
4.1 Project overview 7
4.2 Lab architecture 7
4.3 Detection rules 7
4.4 Create the lab files 7
4.5 The SentinelHash script (sentinelhash.sh) 8
4.6 Create the baseline 9
5. Part D: Test Evidence 11
5.1 Test A: Modified file 11
5.2 Test B: Added file 12
5.3 Test C: Deleted file 12
5.4 Test D: No change (control) 13
5.5 Results summary 13
5.6 Interpretation of results 13
6. Part E: SOC Analyst Finding and Improvement 14
6.1 Analyst finding 14
6.2 Proposed improvement 14

7. Conclusion *14*
References 15

1. Introduction

Modern systems depend on being able to prove that data has not been changed. This report covers the theory behind data integrity and cryptography (Parts A and B), then applies it in a hands-on project called SentinelHash, a file integrity monitor built on Kali Linux (Parts C and D). It closes with a short SOC analyst finding and a proposed improvement (Part E).

1.1 Report structure

Part	Content	Section
A	Integrity, hashing, encryption and digital signatures explained	2
B	SHA-256 vs SHA-1 vs MD5 and a recommendation	3
C	SentinelHash mini-project: design, script and baseline	4
D	Test evidence: modified, added and deleted file detection	5
E	SOC analyst finding and one improvement	6

1.2 Scope and authorization

All practical work used files and systems owned by the author. The lab folder ~/FIM-Lab is separate from Kali Linux and system directories, and every command is a safe, local file operation.

2. Part A: Core Concepts

Draft text for you to review and reword in your own voice, as the assignment asks for your own words.

2.1 Data integrity

Data integrity means that data stays accurate and unchanged unless someone authorized changes it. It is one of the three goals of the CIA triad, alongside confidentiality and availability. Integrity can be lost by accident, for example through disk corruption or a failed transfer, or on purpose, for example when malware edits a configuration file or an attacker alters logs to hide their tracks.

2.2 Hashing

A hash function takes input of any size and produces a fixed-size digest. SHA-256 always outputs 256 bits, written as 64 hexadecimal characters. A cryptographic hash has these useful properties:

- **Deterministic:** the same input always produces the same hash.
- **Avalanche effect:** a one-character change produces a completely different hash.
- **One-way:** the original input cannot realistically be recovered from the hash.
- **Collision resistant:** it should be infeasible to find two different inputs with the same hash.

Hashing does not hide data and uses no key. It acts as a fingerprint, which is why it suits integrity checks.

2.3 Encryption

Encryption protects confidentiality. It transforms readable data (plaintext) into unreadable data (ciphertext) using an algorithm and a key, and the right key reverses it. Symmetric encryption such as AES uses one shared key. Asymmetric encryption such as RSA uses a public and private key pair. Unlike hashing, encryption is meant to be reversible, and on its own it does not prove the data was not altered.

2.4 Digital signatures

A digital signature gives integrity, authenticity and non-repudiation. The sender hashes the message and encrypts that hash with their private key. Anyone with the sender's public key can verify the signature: if the recomputed hash matches, the message is unchanged and came from the key holder. A plain hash cannot prove who created the data, but a signature can.

2.5 Summary comparison

Concept	Main goal	Key used?	Reversible?
Hashing	Integrity (fingerprint)	No	No
HMAC	Integrity + authenticity (shared secret)	Yes, shared	No
Encryption	Confidentiality	Yes	Yes, with the key
Digital signature	Integrity + authenticity + non-repudiation	Yes, key pair	Verifiable, not reversible

3. Part B: SHA-256 vs SHA-1 vs MD5

Property	MD5	SHA-1	SHA-256
Family	MD family (RFC 1321)	SHA-1	SHA-2 (FIPS 180-4)
Digest size	128 bits	160 bits	256 bits
Hex length	32 characters	40 characters	64 characters
Collision attacks	Practical for many years	Practical (SHAttered, 2017)	None known in practice
Status	Broken, do not use for security	Deprecated for security use	Current recommended choice
Typical use today	Non-security checksums only	Legacy systems	Integrity checks, certificates, FIM tools

3.1 Why the older algorithms fail

A hash used for integrity must make it infeasible to craft a different file that has the same hash. For MD5 and SHA-1, researchers have shown practical ways to produce collisions, meaning two different inputs with identical digests. An attacker could use this to replace a file with a malicious one that still matches a stored hash, which defeats the purpose of an integrity check.

3.2 Recommendation

For a new integrity check I would choose SHA-256. It has no known practical collision attacks, it is widely standardised and supported (including by sha256sum on Kali Linux), and it is what real tools such as Wazuh FIM use. The speed difference from MD5 is negligible for file monitoring. SHA-3 or SHA-512 would also be acceptable, but SHA-256 is the best balance of security and compatibility. This is why SentinelHash uses SHA-256.

4. Part C: SentinelHash Mini-Project

4.1 Project overview

SentinelHash is a file integrity monitor that detects added, modified and deleted files in a dedicated lab folder. It uses a Kali Linux terminal, SHA-256 hashing and a CSV change report.

4.2 Lab architecture

1. ~/FIM-Lab holds the monitored files.
2. The SHA-256 baseline is saved as ~/baseline.csv.
3. A current scan is saved as ~/current.csv.
4. The comparison writes the CSV alert report ~/change_report.csv.

The baseline, scan and report files are stored outside FIM-Lab so they are not detected as changes themselves.

4.3 Detection rules

Event	Detection rule
ADDED	Path exists in the current scan but not in the baseline
MODIFIED	Path exists in both, but the SHA-256 hash differs
DELETED	Path exists in the baseline but not in the current scan
No change	All paths and hashes match

4.4 Create the lab files

Open the Kali Linux terminal as a normal user and run:

```
mkdir -p ~/FIM-Lab
echo "approved configuration" > ~/FIM-Lab/critical.txt
echo "training sample" > ~/FIM-Lab/readme.txt
sha256sum ~/FIM-Lab/critical.txt
```

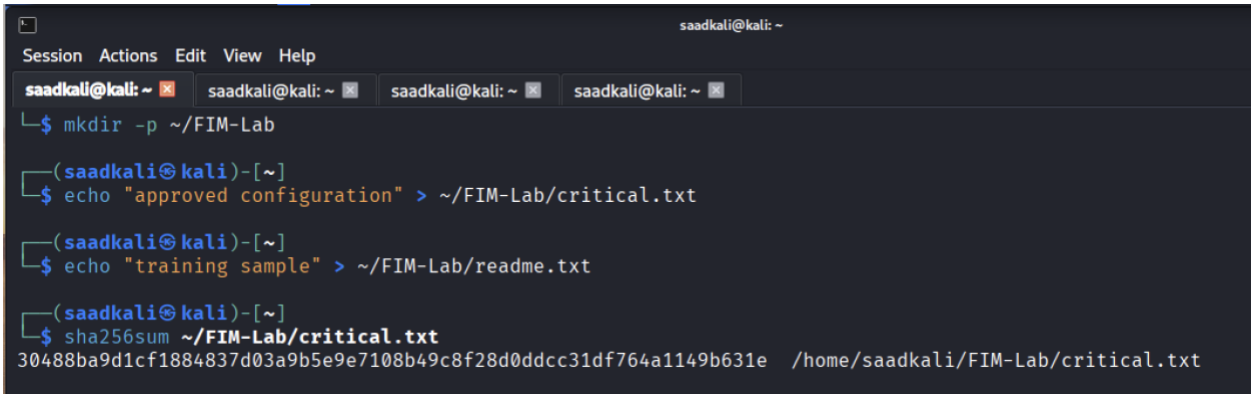


Figure I. Lab directory and sample files.

4.5 The SentinelHash script (sentinelhash.sh)

```
#!/bin/bash
# SentinelHash - simple file integrity monitor (training lab)
root=~/FIM-Lab
base=~/baseline.csv
cur=~/current.csv
report=~/change_report.csv

scan() {
 echo "path,sha256" > "$1"
 find "$root" -type f -exec sha256sum {} + | awk '{print $2","$1}' >> "$1"
}

case "$1" in
 baseline)
  scan "$base"
  echo "Baseline saved to $base"
  cat "$base"
  ;;
 compare)
  scan "$cur"
  echo "status,path,old_hash,new_hash" > "$report"
  awk -F, '
   NR==FNR { if (FNR>1) old[$1]=$2; next }
   FNR>1 {
    seen[$1]=1
    if (!($1 in old))      print "ADDED,"$1",,"$2
    else if (old[$1]!=$2)  print "MODIFIED,"$1","old[$1]","$2
   }
   END { for (p in old) if (!(p in seen)) print "DELETED,"p","old[p]"," }
  ' "$base" "$cur" >> "$report"

  if [ "$(wc -l < "$report")" -eq 1 ]; then
   echo "No changes detected."
  else
   cat "$report"
  fi
  ;;
 *)
  echo "Usage: $0 {baseline|compare}"
```

```
  ;;
esac
```

Make the script executable:

```
chmod +x ~/sentinelhash.sh
```

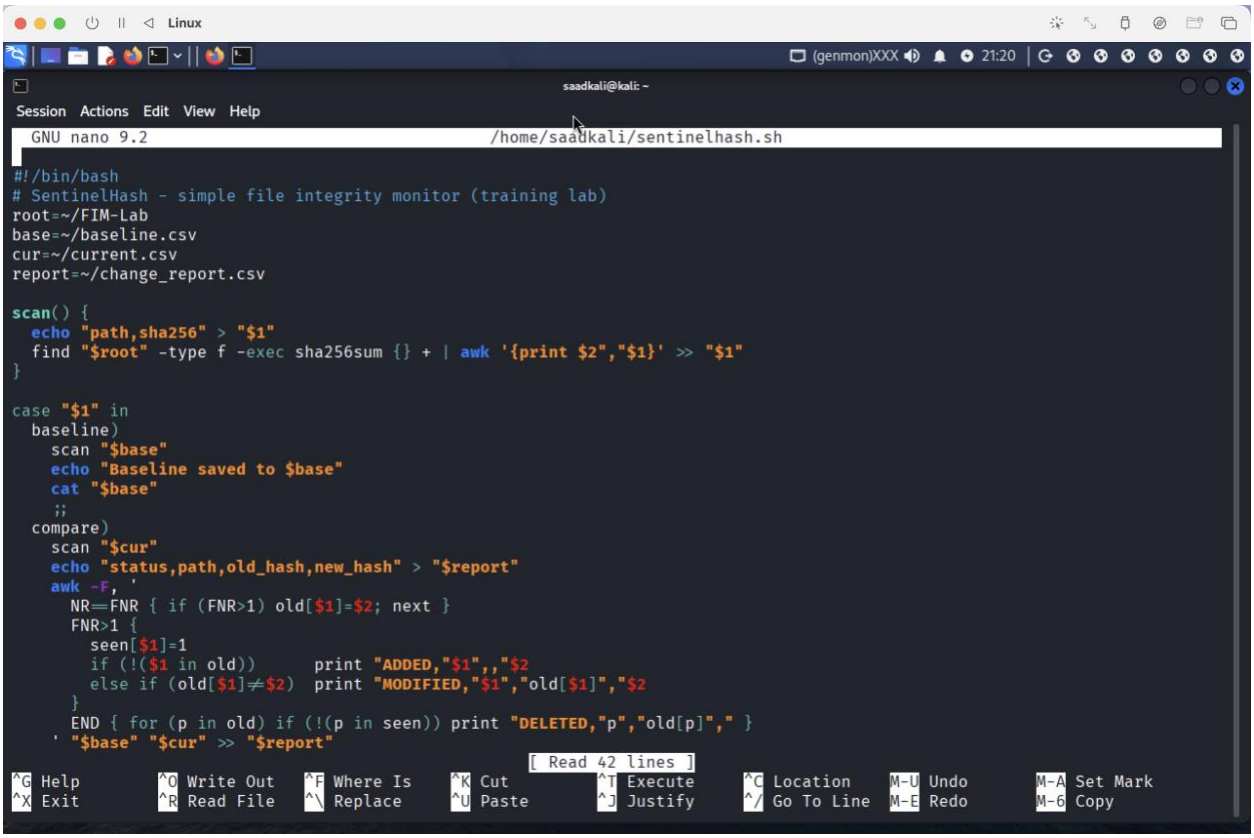




Figure 2. SentinelHash script in the terminal and script executable.

4.6 Create the baseline

A baseline records the expected file state at a chosen time. It was generated before any test changes were made.

```
~/sentinelhash.sh baseline
```



Figure 3. Initial baseline.

5. Part D: Test Evidence

A fresh baseline was generated before each test, only after the previous result had been recorded, so every test is isolated.

5.1 Test A: Modified file

Action: Append text to critical.txt.

Expected result: MODIFIED

```
echo "tampered" >> ~/FIM-Lab/critical.txt
~/sentinelhash.sh compare
cp ~/change_report.csv ~/report_testA.csv
```

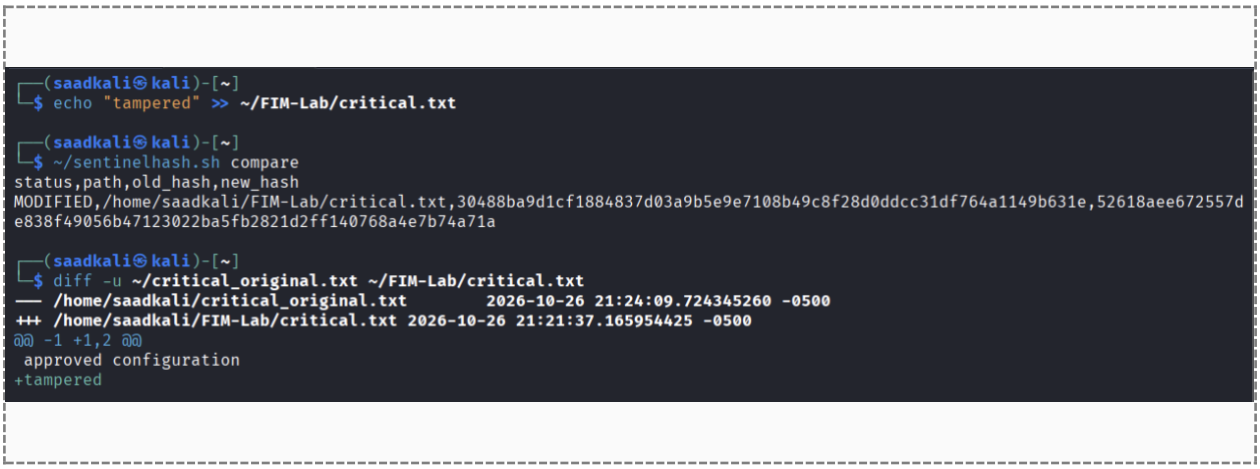


Figure 4. Test A output.

5.2 Test B: Added file

Action: Create new.txt.

Expected result: ADDED

```
~/sentinelhash.sh baseline
echo "new file" > ~/FIM-Lab/new.txt
~/sentinelhash.sh compare
cp ~/change_report.csv ~/report_testB.csv
```

```
┌──(saadkali㉿kali)-[~]
└─$ ~/sentinelhash.sh baseline
Baseline saved to /home/saadkali/baseline.csv
path,sha256
/home/saadkali/FIM-Lab/readme.txt,4ea2a72ed972663f8e56c9680a9e7e5dde08b2a4810e50f5737b88f5a2fc5cae
/home/saadkali/FIM-Lab/critical.txt,52618aee672557de838f49056b47123022ba5fb2821d2ff140768a4e7b74a71a

┌──(saadkali㉿kali)-[~]
└─$ echo "new file" > ~/FIM-Lab/new.txt

┌──(saadkali㉿kali)-[~]
└─$ ~/sentinelhash.sh compare
status,path,old_hash,new_hash
ADDED,/home/saadkali/FIM-Lab/new.txt,,0f15384d18789b1ebf3043dc7b6bc27273c8576373fbeb6f3e15854b588141c0

┌──(saadkali㉿kali)-[~]
└─$ cp ~/change_report.csv ~/report_testB.csv
```

Figure 5. Test B output.

5.3 Test C: Deleted file

Action: Delete readme.txt.

Expected result: DELETED

```
~/sentinelhash.sh baseline
rm ~/FIM-Lab/readme.txt
~/sentinelhash.sh compare
cp ~/change_report.csv ~/report_testC.csv
```

```
┌──(saadkali㉿kali)-[~]
└─$ ~/sentinelhash.sh baseline
Baseline saved to /home/saadkali/baseline.csv
path,sha256
/home/saadkali/FIM-Lab/new.txt,0f15384d18789b1ebf3043dc7b6bc27273c8576373fbeb6f3e15854b588141c0
/home/saadkali/FIM-Lab/readme.txt,4ea2a72ed972663f8e56c9680a9e7e5dde08b2a4810e50f5737b88f5a2fc5cae
/home/saadkali/FIM-Lab/critical.txt,52618aee672557de838f49056b47123022ba5fb2821d2ff140768a4e7b74a71a

┌──(saadkali㉿kali)-[~]
└─$ rm ~/FIM-Lab/readme.txt

┌──(saadkali㉿kali)-[~]
└─$ ~/sentinelhash.sh compare
status,path,old_hash,new_hash
DELETED,/home/saadkali/FIM-Lab/readme.txt,4ea2a72ed972663f8e56c9680a9e7e5dde08b2a4810e50f5737b88f5a2fc5cae,

┌──(saadkali㉿kali)-[~]
└─$ cp ~/change_report.csv ~/report_testC.csv
```

Figure 6. Test C output.

5.4 Test D: No change (control)

Action: Do not modify any file.

Expected result: No content-change event

```
~/sentinelhash.sh baseline
~/sentinelhash.sh compare
```



Figure 7. Test D output.

5.5 Results summary

Test	Action	Expected	Actual	Pass / Fail
A	Modify critical.txt	MODIFIED	MODIFIED	PASS
B	Create new.txt	ADDED	ADDED	PASS
C	Delete readme.txt	DELETED	DELETED	PASS
D	No change	No event	No event	PASS

5.6 Interpretation of results

The SentinelHash tests successfully demonstrated the detection of file integrity changes. Test A detected critical.txt as **MODIFIED**, Test B classified new.txt as **ADDED**, and Test C identified readme.txt as **DELETED**. Test D detected no changes when the files remained unchanged from the baseline. These results show that comparing SHA-256 hashes and file paths against a trusted baseline can identify file modifications, additions, and deletions, because even a small change in file contents produces a different hash.

6. Part E: SOC Analyst Finding and Improvement

6.1 Analyst finding

Write this from your real Test A to C results. Use the structure below.

Field	Entry
Alert type	File integrity change
Affected file(s)	~/FIM-Lab/critical.txt
Event	MODIFIED / ADDED / DELETED
Evidence	Old hash vs new hash from report
Assessment	Authorized change or possible tampering, and why
Recommended action	investigate who changed it, restore from known-good copy

The SentinelHash results confirmed that the monitoring system could identify changes to the monitored files. Test A detected a hash mismatch in critical.txt, Test B identified the newly created new.txt, and Test C flagged the missing readme.txt. Each event matched its expected classification, demonstrating how SHA-256 hashing and baseline comparison help detect file modifications, additions, and deletions. Although these results indicate changes to file integrity, further investigation would be needed to determine whether the activity was legitimate or unauthorized.

6.2 Proposed improvement

The main weakness of SentinelHash is that the baseline is a plain file. An attacker with write access could edit baseline.csv to match their changes, and the monitor would report nothing. A practical improvement is to protect the baseline's integrity:

- Make the baseline read-only (chmod 400) and store a copy on a separate, write-protected location.
- Sign the baseline, for example with an HMAC or a GPG signature, and verify the signature before every comparison.
- Automate scheduled scans (for example with cron) and send alerts, so changes are caught without manual runs.

One improvement

I would improve SentinelHash by using a GPG digital signature to protect the baseline. GPG (GNU Privacy Guard) is a cryptographic tool that creates digital signatures using a private key and verifies them using a public key. Before each comparison, SentinelHash would verify the signature to check whether the baseline has been changed. If the signature verification fails, the monitor would raise an alert instead of trusting the baseline. This would help prevent attackers from modifying the baseline to hide unauthorized file changes.

7. Conclusion

In conclusion, SentinelHash was developed as a Bash-based File Integrity Monitoring tool that uses SHA-256 hashes to monitor files. The tests demonstrated its ability to identify modified files, newly added files, and deleted files by comparing current scans with a saved baseline. The results showed that even a small change in a file's contents produces a different SHA-256 hash, making hashing useful for detecting unauthorized modifications. The exercise also highlighted the importance of protecting the baseline, as an attacker could manipulate it to hide file changes.

References

- NIST FIPS 180-4, Secure Hash Standard (SHS).
- RFC 1321, The MD5 Message-Digest Algorithm.
- Stevens et al., The first collision for full SHA-1 (SHAttered), 2017.